

הצעת פרויקט גמר

תאריך: 07/12/2025

לידיעת: יחידת הפרויקטים - מה"ט

בהתאם להנחיות חוזר מנהל מה"ט 11-4-51 נספח מס' 1.

1. פרטי הסטודנטים

שם מלא	תעודת זהות	כתובת מגורים	טלפון נייד	תאריך סיום לימודים משוער
אופיר גואטה	326091337	רזניק דויד 7, נתניה	0586262800	-
איתי בר	208731323	ביאליק 10, בית שמש	0512087313	-

2. פרטי המכללה

שם המכללה: בסמ"ח

סמל המכללה: 71605

מסלול ההכשרה: טכנאים מוסמכים

מגמת לימוד: הנדסת תוכנה

מקום ביצוע הפרויקט: בסמ"ח

3. פרטי המנחה האישי

שם המנחה	עילי גילת
כתובת	האירוסים 67 קדימה צורן
טלפון נייד	052-6010916
תואר	מדעי המחשב, מכללת נתניה
מקום עבודה/תפקיד	צה"ל

* דיפלומת המנחה מצורפת בסוף המסמך

חתימת הסטודנט  חתימת המנחה האישי  חתימת הגורם המקצועי מטעם מה"ט 

4. שם הפרויקט

Classly

5. רקע

עולם החוגים בישראל צומח ומתפתח משנה לשנה. יותר ויותר אנשים מחפשים מקום שבו יוכלו לשלב פעילות גופנית, אמנות ותנועה בחיי היומיום. עם זאת, מאחורי הקלעים מתנהל תהליך מורכב שמתנהל בצורה ידנית ומפוזרת. דבר זה יוצר קשיים הן מצד התלמידים, שנדרשים להתעסק ברישומים ותשלומים מסורבלים, והן מצד המורים והמנהלים, שנאלצים להשקיע זמן רב בארגון במקום להתמקד בהוראה ובפיתוח

הקהילה.

תהליכים אלו גוזלים זמן יקר מהמורים ומהמנהלים, פוגעים ביעילות ואף מקשים על התלמידים.

מעבר לכך, פעילות ספורטיבית ואמנותית היא לא רק עניין של בריאות פיזית. היא משפיעה בצורה ישירה על הנפש, יוצרת תחושת שייכות ותורמת לחוסן חברתי. לכן, יצירת מערכת דיגיטלית מאוחדת שתפשט ותיעל את כל תהליכי הניהול בסטודיו היא לא רק פתרון טכנולוגי, אלא גם מענה חברתי וקהילתי חשוב.

6. מטרת המערכת

המערכת נועדה לספק פתרון דיגיטלי מלא לניהול חוגים וסטודיו, החל מהרשמות ותשלומים, דרך ניהול מערכות שעות ועד מעקב נוכחות. המטרה היא לאחד את כל התהליכים תחת פלטפורמה אחת נוחה, יעילה ושקופה עבור מנהלים, מורים ותלמידים.

7. סקירת מצב קיים בשוק ובעיות קיימות

- כיום, ניהול חוגים מתבצע לרוב בדרכים חלקיות ומסורבלות:
- רישום ותשלומים ידניים או בהודעות פרטיות.
 - מערכות שעות שאינן מרוכזות וקלות לשינוי ללא תיאום.
 - חוסר שקיפות בתשלומים הן מול הסטודיו והן מול המורים.
 - היעדר כלי דיגיטלי לניהול נוכחות.

התוצאה: עומס על המנהלים, חוסר סדר, טעויות אנוש, ותחושת בלגן שפוגעת גם ביעילות הסטודיו וגם בחוויית המשתמש.

8. מה הפרויקט אמור לחדש או לשפר

אוטומציה של תהליכים.

רישום ותשלומים מקוונים, ניהול מערכת שעות ונוכחות בצורה מרוכזת.

שיפור חוויית המשתמש.

ממשק אינטואיטיבי ופשוט ל תלמידים, מורים ומנהלים.

ייעול הניהול הפנימי.

פלטפורמה אחת לכלל הצרכים במקום שימוש בכלים מפוזרים.

תרומה קהילתית.

חיזוק תחום האמנות והספורט בארץ, עידוד פעילות גופנית ויצירת חיבור בין אנשים.

9. דרישות מערכת ופונקציונליות

9.1 דרישות מערכת:

- המערכת תשמש בראש ובראשונה את מנהל הסטודיו, אך גם מורים ותלמידים ייהנו מממשקים מותאמים.
- על המערכת לתמוך בכמות גדולה של משתמשים במקביל (עונות רישום, ימי תשלום).
- על המערכת לאפשר עדכון מהיר של מערכות שעות, תוך שמירה על עקביות במידע לכל המשתמשים.
- המערכת תהיה נגישה מכל מכשיר (מחשב, טאבלט, סמארטפון).
- על המערכת לשמור מידע באופן מאובטח (פרטי תלמידים, פרטי תשלום).

9.2 דרישות פונקציונליות:

מסך התחברות - כולל הרשמה ובחירת סוג משתמש (מנהל / מורה / תלמיד).

צד מנהל:

- ניהול תלמידים והרשמות.
- ניהול תשלומים (לסטודיו ולמורים).
- בנייה וניהול של מערכת שעות מרכזית.
- צפייה בדוחות (נוכחות, כספים, הרשמות).

צד מורה:

- מערכת שעות אישית.
- רשימת תלמידים לשיעור + אפשרות למלא נוכחות.
- מעקב אחר תשלומים המגיעים לו.

צד תלמיד:

- הרשמה מקוונת לחוגים ותשלום אונליין.
- גישה למערכת שעות אישית.
- קבלת עדכונים על שינויים בשיעורים.

10. בעיות צפויות במהלך הפיתוח ופתרונות

10.1 בעיות צפויות:

- עומסים על המערכת בתקופות רישום ותשלום.
- ניהול שינויים במערכת שעות העלול לבלבל תלמידים ומורים.
- שמירת מידע רגיש (פרטי תשלום ותלמידים).
- ניהול מספר סטטוסים במקביל.

10.2 פתרונות:

- שימוש ב-Kubernetes (Auto Scaling) לניהול עומסים.
- יצירת מנגנון התראות ועדכונים אוטומטיים במקרה של שינוי מערכת שעות.
- הצפנת נתונים, שימוש ב-SSL ועמידה בתקנים לניהול מידע ותשלומים.

11. פתרון טכנולוגי נבחר

המערכת היא מערכת וובית המורכבת מצד לקוח (Client), צד שרת (Server) ומסד נתונים (DB). המערכת תיפרס באמצעות תהליך CI/CD לשרתים.

טכנולוגיות בשימוש:

רכיב	טכנולוגיה/שירות
Front End	(React + Next.js) SSR
Backend	Convex Backend
DB	Convex DB
Auth	Clerk
Payments	Transilla
Sync Engine	מנגנון סנכרון בזמן אמת בין DB ל-Frontend

12. שפות הפיתוח

שפת TypeScript היא שפת הפיתוח המרכזית במערכת ומשמשת בכל הרכיבים - בצד הלקוח, בצד השרת ואינטגרציות החיצוניות. היא משלבת את הפשטות של JavaScript עם מערכת טיפוסים שמונעת שגיאות ומיעלת את העבודה.

ב-Front End היא משמשת עם React ליצירת ממשקים אינטראקטיביים.
ב-Next.js היא משמשת לניהול רינדור בצד השרת ו-API.
בצד ה-Backend היא משמשת עם Convex לכתיבת לוגיקה עסקית ולתקשורת עם מוד הנתונים Convex DB.
בנוסף, היא משמשת למנוע הסנכרון ולעבודה עם שירותים חיצוניים כמו Clerk ו-Transilla.

13. תיאור הארכיטקטורה הנבחרת

הארכיטקטורה שנבחרה מבוססת על הפרדה ברורה בין שכבות התוכנה ומשלבת טכנולוגיות מתקדמות לביצועים, אבטחה וניהול נתונים יעיל.
הממשק נבנה ב-React וב-Next.js עם רינדור בצד השרת (SSR) לקבלת חוויית משתמש עשירה וזמני טעינה מהירים.
הבקאנד פועל על Convex, המספק עיבוד, סנכרון בזמן אמת ותקשורת ישירה עם Convex DB.
המערכת כוללת אינטגרציה עם שירותי אימות של Clerk, תשלומים מאובטחים ב-Transilla וניהול משימות מתוזמנות.
הארכיטקטורה תומכת בפיתוח מהיר, אבטחה מובנית וביכולת התרחבות קלה לאורך זמן.

14. חלוקה לתכניות ומודולים

צד המערכת	רכיב	תיאור
צד שרת	Models	ייצוג מבנה הטבלאות במסד הנתונים (שיעור, מסלול, מערכת שעות, סטודיו, קטגוריות, משתמשים לפי סוגים).
	Repositories	אחראים על פעולות CRUD (יצירה, קריאה, עדכון, מחיקה) מול מסד הנתונים.
	Services	אחראים על לוגיקה עסקית ושמירת מידע במסד הנתונים.
צד לקוח	מסכים	מסכי התחברות, בית, קטגוריות, שיעור, סיכום, תשלום, מסכי מורה ומנהל.
	אלגוריתם	אלגוריתם הרשמה חכמה.
	תצוגות	מערכת שעות כוללת, סטטיסטיקות ותשלומים.

15. השרת

השרת יתבסס על ארכיטקטורת Serverless ויפעל בסביבת ענן מלאה באמצעות Vercel, התומכת ב-Next.js וב-API endpoints. רכיב ה-Backend מבוסס על Convex, הפועל גם הוא במודל Serverless ומנהל את הלוגיקה העסקית וה-DB בזמן אמת בענן, כולל גיבויים וניהול עומסים אוטומטי.

16. ממשק המשתמש / GUI

הלקוח ניגש למערכת דרך הדפדפן במחשב או בטלפון נייד, ללא צורך בהתקנת אפליקציה. המערכת נבנית בטכנולוגיית React עם תמיכה מלאה ברספונסיביות ועיצוב מודרני.

הגישה למערכת מתבצעת באמצעות כתובת ייעודית המאוחסנת בענן, והאימות נעשה באמצעות שירות Clerk.

17. ממשקים למערכות אחרות / API

המערכת מתממשקת לשירותים חיצוניים באמצעות API: Tranzilla - שירות סליקה לניהול תשלומים מאובטחים, פריסת תשלומים והפקת קבלות. Clerk - שירות אימות משתמשים לניהול חשבונות, הרשאות ו-Session. המערכת בנויה במבנה מודולרי המאפשר הוספת ממשקים נוספים (CRM, BI), מערכות דיור ועוד).

18. שימוש בחבילות תוכנה

צד	חבילה	תיאור / ייעוד
שרת	Convex	ניהול DB ו-Backend בזמן אמת
	Clerk	ניהול הרשאות ומשתמשים
	Axios / Fetch	קריאות API
	חבילות (SDK) Tranzilla או (REST)	אינטגרציית סליקה
לקוח	mui/material@	ספריית עיצוב מודרנית
	reduxjs/toolkit@	ניהול state מרכזי
	tanstack/react-query@	ניהול קריאות API וסנכרון נתונים
	react-toastify	התראות למשתמש
	Formik	ניהול טפסים
	react-dropzone	גרירת קבצים
	framer-motion	אנימציות

19. שימוש במבני נתונים וארגון קבצים

Convex DB משמש כמאגר הנתונים המרכזי, עם אוספים המייצגים ישויות כגון: משתמשים, שיעורים, קטגוריות, תשלומים ונוכחות, עם קשרים לוגיים ביניהם (Student → Enrollment → Lesson → Category). הנתונים נשמרים בענן, מגובים אוטומטית ותומכים בסנכרון בזמן אמת. קבצי המערכת מאורגנים לפי שכבות: Components, Pages, Services, Models, Repositories וכו'.

20. תרשימי מערכת מרכזיים

Use Case - פירוט מקרי שימוש עיקריים למשתמשי המערכת (הורה/תלמיד, מורה, מנהל).

Sequence Diagram - תיאור רצף הפעולות בין Client, Server ו-DB (לדוגמה: הרשמה רפואית/הרשמה לחוג).

Data Flow - תרשים זרימת נתונים בין רכיבי המערכת ומסד הנתונים.

21. תיאור המרכיב האלגוריתמי - חישובי

אלגוריתם הרשמה חכמה למסלול אחראי להתאים לכל תלמיד את המסלול האופטימלי של שיעורים לפי העדפותיו האישיות (יום, שעה, מיקום, רמה). המערכת מנתחת את כלל השיעורים הקיימים, מסננת התנגשויות בזמנים ובמרחקים, ומדרגת את המסלולים האפשריים לפי התאמה וזמינות. התוצאה היא הצגת מספר מסלולים מומלצים לתלמיד, המספקים שילוב מיטבי בין נוחות, גמישות וזמינות.

22. תיאור/התייחסות לנושאי אבטחת מידע

המערכת תשלב מנגנוני אבטחה מתקדמים ברמת המשתמש, השרת והנתונים. האימות יתבצע באמצעות שירות Clerk, המנפיק Access Tokens המשמשים לגישה מאובטחת ל-Convex. במערכת יוגדרו RBAC (Roles) המגבילים גישה לפעולות בהתאם להרשאות המשתמש. כל הבקשות לשרת יעברו אימות ובקרה בצד השרת בלבד.

המערכת תעבוד בפרוטוקול HTTPS, תכלול מדיניות CORS מבוקרת, ולוגים ישמרו לפעולות קריטיות לצורך ביקורת וניטור.

23. משאבים הנדרשים לפרויקט

23.1 מספר שעות עבודה וחלוקת עבודה:
הפרויקט מתומחר בכ-350 שעות עבודה.
חלוקה:

משימה	אחראים
תכנון מסד נתונים ואלגוריתם הרשמה	אופיר
פיתוח צד שרת (Next.js + Convex)	אופיר
פיתוח צד לקוח (React)	איתי
עיצוב UX/UI	איתי
התממשקות לשירותים חיצוניים	אופיר
בדיקות מערכת והרצה	איתי ואופיר

23.2 ציוד נדרש:

- עמדות מחשב עם Windows / macOS וחיבור אינטרנט.
- מכשירי מובייל לבדיקת רספונסיביות.

23.3 תוכנות נדרשות:

Tool	Purpose
Visual Studio Code	Code editor
Git + GitHub	Version control and hosting
Node.js + npm	JavaScript runtime and package manager
Google Chrome	Web browser and developer tools
Convex Console	Convex database interface/management
Figma	UI/UX design tool
Transilla Dashboard	(Likely) Transifex/translation management dashboard

23.4 ידע חדש נדרש:

- עבודה עם Convex.
- שימוש ב-Clerk לניהול הרשאות.
- חיבור ל-Transilla לצורך סליקה.
- שימוש ב-SSR (Next.js).
- שימוש ב-Tailwind CSS.
- תכנון ויישום אלגוריתם הרשמה חכם.

23.5 ספרות ומקורות מידע:

- דוקומנטציה של React, Next.js, Convex.dev, Clerk.com, Transilla, Tailwind CSS, Figma.

24. תוכנית עבודה ושליבים למימוש הפרויקט

שלב	תיאור משימה	הערכת זמן (שעות)
שלב 1: תכנון והגדרות		
1.1	בניית לוח ניהול משימות (Sprint-ים והקצאת משימות)	5
1.2	תכנון בסיס הנתונים (DB)	20
1.3	לימוד והעמקה בטכנולוגיות הנדרשות לפרויקט	30
1.4	תכנון ארכיטקטורה וזרימת נתונים כללית	20
שלב 2: תשתית ואבטחה		
2.1	הקמת מנגנון הרשאות והתחברות משתמשים	20
שלב 3: עיצוב וחווית משתמש (UI/UX)		
3.1	עיצוב ושרטוט ראשוני של מסכי המערכת	15
3.2	עיצוב ותכנון UI/UX כולל של המערכת	10
שלב 4: פיתוח מסכי ליבה		
4.1	פיתוח מסך בית וניהול קטגוריות	25
4.2	פיתוח מסך שיעור והטמעת מנגנון הרשמה חכמה	35

25	פיתוח אלגוריתם הרשמה חכמה	4.3
20	פיתוח מסך סיכום ותשלום מאובטח	4.4
		שלב 5: פיתוח דפי משתמשים וניהול
15	פיתוח דפי מערכת שעות (תלמיד, מורה, מנהל)	5.1
25	פיתוח דף מורה (ממשק ניהול אישי)	5.2
45	פיתוח דפי מנהל (ניהול שיעורים, מורים, קטגוריות, תשלומים, סטטיסטיקות)	5.3
		שלב 6: בדיקות והטמעה
15	בדיקות יחידה ואינטגרציה (Unit & Integration Tests)	6.1
10	בדיקות קצה ושיפורים (End-to-End Testing & Refinements)	6.2
20	העלאה ל-Production והגדרת CI/CD	6.3
כ-340 - 350 שעות	סה"כ שעות עבודה משוער	סך הכל

25. תכנון הבדיקות שיבוצעו

25.1 בדיקות תהליכיות (Full Flow):

בדיקה תהליכית	תיאור הבדיקה
הרשמה למערכת כמשתמש חדש	בדיקת תהליך ההרשמה המלא למערכת.
התחברות עם פרטים שגויים	בדיקת טיפול המערכת בניסיון התחברות עם שם משתמש או סיסמה לא נכונים.
הצגת קטגוריות בדף הבית	בדיקת הצגה נכונה ומלאה של קטגוריות השיעורים בדף הבית.
הרשמה לשיעור	בדיקת תהליך הרישום של משתמש לשיעור ספציפי.
תשלום מאובטח והפקת קבלה	בדיקת תהליך התשלום המאובטח והפקת מסמך קבלה למשתמש.
דיווח נוכחות על ידי מורה	בדיקת היכולת של מורה לסמן נוכחות של תלמידים בשיעור.
הפקת דו"ח סטטיסטי על ידי מנהל	בדיקת היכולת של מנהל להפיק דו"ח המכיל נתונים סטטיסטיים רלוונטיים.

25.2 בדיקות יחידה (Unit Tests):

תהליך	בדיקות אימות נתונים/תקינות
אימות נתוני הרשמה	שדות חובה, אימות מייל וסיסמה.
יצירת שיעור חדש	שדות חובה, אי-חפיפה בזמנים.
הרשמה לשיעור	בדיקת תפוסה, כפילות רשומה.
תהליך תשלום	בדיקת תקינות שדות ומניעת חיוב כפול.

26. בקרת גרסאות (Version Control)

ניהול גרסאות יתבצע באמצעות Git, עם אחסון ב-GitHub. השימוש ב-Git מאפשר עבודה שיתופית נוחה, מעקב אחר שינויים, יצירת Branchים לפיצ'רים שונים ומניעת התנגשויות בקוד.



חתימת המנחה האישי



חתימת הסטודנט

נספחים: דיפלומת מנחה, תרשימי מערכת, מסכי מערכת לדוגמה ועוד.